

Critical Regions

Definition

A **critical region**, also called a **critical section**, is the part of a program where a process accesses shared memory, shared files, or other shared resources that can lead to race conditions.

1. Why Critical Regions Matter

Main Idea

Race conditions are avoided by making sure that two processes do not read or write the same shared data at the same time.

Key Message

The central requirement is **mutual exclusion**: if one process is using a shared variable, file, or resource, other processes must be excluded from using that same shared item until the first process is finished.

In the print spooler example, the race happened because process B started using the shared variable `in` before process A had finished using it. A correct solution must prevent that kind of overlap.

2. Internal Work vs. Critical Work

Normal Execution

Most of the time, a process performs private computations that do not affect other processes. These instructions do not need special protection because they do not touch shared data.

Critical Execution

When a process accesses shared memory, shared files, shared variables, or shared devices, it enters a dangerous part of the program. This part must be controlled carefully to avoid races.

Simple View

A process can be thought of as alternating between two kinds of work:

- **Noncritical work**: private computation with no shared-data risk.
- **Critical region**: shared-data access where mutual exclusion is required.

3. The Goal

Mutual Exclusion Goal

If one process is inside its critical region, no other process should be allowed to enter its own critical region for the same shared resource.

If this rule is enforced, then the shared data is accessed by only one process at a time. That prevents the timing problem where one process sees stale or half-updated data from another process.

Important Distinction

Preventing two processes from being in their critical regions at the same time avoids race conditions, but a complete solution must also be fair and efficient.

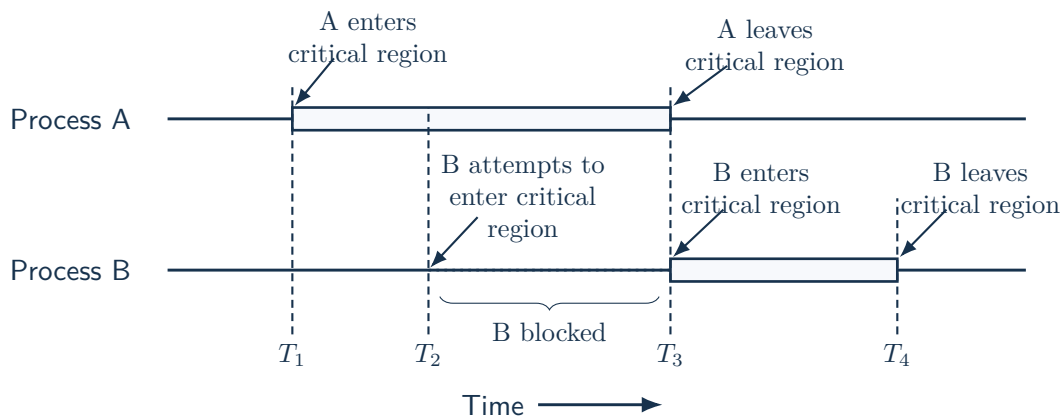
4. Four Requirements for a Good Solution

No.	Requirement	Meaning
1	No two processes may be simultaneously inside their critical regions.	This is the mutual exclusion rule. Only one process may access the protected shared data at a time.
2	No assumptions may be made about speeds or the number of CPUs.	The solution must work whether processes run slowly, quickly, on one CPU, or on many CPUs.
3	No process running outside its critical region may block any process.	A process that is not using the shared data should not prevent others from entering their critical regions.
4	No process should have to wait forever to enter its critical region.	The solution must avoid starvation. Every waiting process should eventually get a turn.

5. Timing Diagram

What the Diagram Shows

Process A enters its critical region first. Process B tries to enter while A is still inside, so B is blocked. When A leaves, B enters immediately. Later, B leaves and both processes are outside their critical regions again.



6. What Happens at Each Time

Time	Event
T_1	Process A enters its critical region and starts using the shared resource.
T_2	Process B attempts to enter its critical region, but it is not allowed because A is already inside.
T_2-T_3	Process B is blocked. It must wait until the shared resource becomes available.
T_3	Process A leaves its critical region, so process B can enter immediately.
T_4	Process B leaves its critical region. Now no process is inside a critical region.

7. Why the Four Requirements Matter

Requirement 1: Mutual Exclusion

This is the basic safety rule. Without it, two processes can modify shared data at the same time and create a race condition.

Requirement 2: No Timing Assumptions

A correct operating-system solution cannot depend on one process being faster than another. It also cannot depend on having exactly one CPU.

Requirement 3: No Outside Blocking

If a process is doing private work outside its critical region, it should not stop another process from entering a critical region. Otherwise, the system wastes parallelism.

Requirement 4: No Starvation

A process that wants to enter its critical region should eventually be allowed to enter. It should not wait forever while other processes repeatedly get access.

8. Common Terms

Term	Meaning
Race condition	A bug where the result depends on the timing of concurrent access to shared data.
Critical region	The part of the program that accesses shared resources and must be protected.
Mutual exclusion	A rule that allows only one process at a time to use a shared resource.
Blocked process	A process that cannot continue because it is waiting for a resource or condition.
Starvation	A situation where a process waits forever and never gets access to the resource it needs.

9. Quick Review

Key Points

- A critical region is code that accesses shared data or shared resources.
- Mutual exclusion prevents more than one process from entering a protected critical region at once.
- Race conditions can be avoided if no two processes are in their critical regions at the same time.
- A full solution must also be fair and efficient.
- Correctness must not depend on process speed or the number of CPUs.
- A process outside its critical region should not block another process.
- No process should wait forever to enter its critical region.